

MANITOR AI: Personal Life Operating System

Complete System Documentation

1. Project Overview

MANITOR AI is a full-stack, AI-powered "Personal Life Operating System." It is designed as a centralized command center to manage work tasks, financial expenses, calendar events, academic studies, personal notes, and long-term goals. At its core sits MONI AI—an intelligent virtual assistant powered by Google Gemini and long-term vector memory—capable of parsing natural language to inject data directly into the system databases in real-time.

1.1 Architecture

The architecture follows a modern decoupled client-server model:

- **Frontend:** Next.js 15 (React 19) with Turbopack, styled using pure CSS (glassmorphism/cyberpunk aesthetic) and Lucide React icons.
- **Backend:** Python FastAPI, offering extremely fast asynchronous endpoints and WebSocket connections.
- **Relational Database:** PostgreSQL (Neon Database) managed via SQLAlchemy ORM.
- **Vector Database:** Qdrant, running in a Docker container, used for storing semantic embeddings to give the AI long-term memory retrieval.
- **Hosting:** Frontend deployed on Vercel Edge Networks. Backend deployed on Render via Docker.

2. Directory Structure

manitor-ai/	
├── frontend/	# Next.js Application
│ ├── src/	
│ │ ├── app/	# Next.js App Router (Pages & Layouts)
│ │ │ ├── analytics/	# Financial & Productivity Analytics
│ │ │ ├── calendar/	# Schedule & Events UI
│ │ │ ├── expenses/	# Financial Tracking UI
│ │ │ ├── goals/	# Long-term OKR Tracking UI
│ │ │ ├── login/	# JWT Authentication UI
│ │ │ ├── notes/	# Personal Knowledge Base UI
│ │ │ ├── study/	# Academic & Learning UI
│ │ │ ├── tasks/	# Actionable Todo UI
│ │ │ ├── work/	# Career & Professional UI
│ │ │ ├── globals.css	# Core Design System (CSS Variables)
│ │ │ ├── layout.tsx	# Global App Wrapper
│ │ │ └── page.tsx	# Main Dashboard
│ │ └── components/	# Reusable React Components (Sidebar,

├── lib/	# Utilities (API Interceptors)
├── next.config.ts	# Next.js Build Configuration
├── package.json	# Frontend Dependencies
├── backend/	# FastAPI Application
│ ├── routers/	# API Endpoints
│ │ ├── ai.py	# WebSocket connection to Gemini & Co
│ │ ├── analytics.py	# Data aggregation for charts
│ │ ├── auth.py	# JWT generation and validation
│ │ ├── calendar.py	# Events CRUD
│ │ ├── expenses.py	# Finances CRUD
│ │ ├── goals.py	# OKR CRUD
│ │ ├── notes.py	# Notes CRUD & Qdrant Embedding Inser
│ │ ├── study.py	# Academic CRUD
│ │ └── work.py	# Tasks CRUD
│ ├── database.py	# SQLAlchemy Engine & Session Configu
│ ├── main.py	# FastAPI Application Entrypoint & CO
│ ├── models.py	# SQLAlchemy Database Table Schemas
│ ├── schemas.py	# Pydantic Validation Models
│ ├── websockets_manager.py	# Global WebSocket broadcasting
│ ├── Dockerfile	# Render Deployment Instructions
│ └── requirements.txt	# Python Dependencies
├── docker-compose.yml	# Local Multi-Container Orchestration
└── README.md	# High-level overview

3. Core Features & Capabilities

3.1 AI Command Injection (The "Moni" Engine)

The core feature of this platform is the AI Agent's ability to execute actions. When a user sends a message via the WebSocket (`/api/v1/ai/ws/chat`), the backend feeds the prompt to Google Gemini along with the user's current life context (open tasks, recent expenses, etc.). If the user asks the AI to log an expense or add a task, Gemini outputs a hidden system command (e.g., `||CMD_ADD_EXPENSE: {"amount": 50, "category": "Food"}||`). The FastAPI backend intercepts this stream, strips the command from the user interface, parses the JSON, and instantly commits the action to the PostgreSQL database.

3.2 Real-time Dashboard

The frontend `Dashboard` component aggressively fetches statistics across all modules (Total Expenses, Active Tasks, Knowledge Base Size) and displays them using custom animated UI components. It features a glowing, futuristic aesthetic built entirely in custom CSS.

3.3 JWT Authentication

Security is enforced using JSON Web Tokens.

- **Login:** The user provides credentials (Hardcoded to `Manikandan / 63798` in `auth.py`).
- **Token:** The backend issues an HS256 JWT valid for 7 days.

- **Authorization:** The frontend stores the token in `localStorage` and attaches it as a `Bearer` header to all API requests. The backend `Depends (security)` validates it before granting access to protected routes.
-

4. API Endpoints Reference

Route	Method	Description
<code>/api/v1/auth/login</code>	POST	Authenticates user and returns JWT token.
<code>/api/v1/ai/ws/chat</code>	WS	WebSocket endpoint for real-time AI streaming and command parsing.
<code>/api/v1/expenses/</code>	GET/ POST	Retrieve all expenses / Log a new expense.
<code>/api/v1/work/</code>	GET/ POST	Retrieve all tasks / Create a new work task.
<code>/api/v1/notes/</code>	GET/ POST	Retrieve notes / Create note and generate Qdrant vector embedding.
<code>/api/v1/analytics/dashboard</code>	GET	Retrieve aggregated dashboard statistics.

5. Environment Variables Configuration

To run this application, the following environment variables **MUST** be configured.

Backend (`backend/.env` & Render Dashboard)

- `DATABASE_URL`: Connection string for PostgreSQL (e.g., Neon Database). Example: `postgresql://user:pass@ep-host.neon.tech/neondb`
- `GEMINI_API_KEY`: API key from Google AI Studio to power the intelligence engine.
- `JWT_SECRET`: A secure random string used to sign authentication tokens.
- `QDRANT_URL`: URL to the Qdrant Vector database. Example: `http://manitor-qdrant:6333`

Frontend (`frontend/.env.local` & Vercel Dashboard)

- `NEXT_PUBLIC_API_URL`: The absolute URL pointing to the FastAPI backend. Example: `https://manitor-backend.onrender.com`
-

6. Deployment Guide

6.1 Database (Neon Tech)

1. Create a Postgres database on Neon.tech.
2. Copy the connection string and place it in the Backend's `DATABASE_URL`.
3. *Note: The FastAPI application will automatically create all required tables upon boot (`models.Base.metadata.create_all(bind=engine)`).*

6.2 Vector Database (Qdrant on Render)

1. Deploy a new "Web Service" on Render.
2. Select "Deploy an existing image from a registry".
3. Image URL: `qdrant/qdrant`.
4. Copy the generated URL and place it in the Backend's `QDRANT_URL`.

6.3 Backend (FastAPI on Render)

1. Deploy a new "Web Service" on Render.
2. Connect the GitHub repository and set the Root Directory to `backend` (or use Docker runtime if building from root).
3. Ensure `DATABASE_URL`, `GEMINI_API_KEY`, and `QDRANT_URL` are added to the Environment Variables.
4. Deploy.

6.4 Frontend (Next.js on Vercel)

1. Import the GitHub repository into Vercel.
 2. Set **Framework Preset** to `Next.js`.
 3. Set **Root Directory** to `frontend`.
 4. Add `NEXT_PUBLIC_API_URL` to Environment Variables.
 5. Deploy.
-

7. Automated Testing Framework

The system utilizes an aggressive CI-style automated testing suite:

- **Backend (Pytest):** Located in `backend/tests/`. Tests JWT issuance, PostgreSQL insertion logic, and WebSocket interaction without touching production data. Run via `pytest`.
- **Frontend (Jest):** Located in `frontend/__tests__/`. Validates DOM rendering, asynchronous state updates, and dynamic CSS injection utilizing React Testing Library. Run via `npm run test`.